

前言

应上一篇[从源码吃透共识协议：raft 日志复制 - 知乎\(zhihu.com\)](#)文章中挖的坑，我将我在论文，开源库的文档，各博客文章中搜罗到的一些 raft 的优化方案做了详细的总结，产出了这篇文章。

对于 raft 优化的点大致有三个方向：Leader Election, [Log Replication](#), [Cluster Member Change](#), 限于篇幅，这篇文章先讲**六种 raft 在 Leader Election 上的优化**，其中绝大部分优化在具体实现时，**几乎都是必要的**，我在好几个 raft 的权威开源库中都见过相关的实现（etcd, tikv, braft, [nuraft](#) 等）

有的优化手段在 raft 大论文中也有提到，但鉴于是一大部分人都没时间啃那 200 多页的论文，直接从我这一系列文章中快速吸收是个不错的选择。

另外，我之前把我看过的比较优质的文档和文章链接做了一个汇总，由于一直在学习阶段，所以那篇文章也会一直更新：

共识协议是难度较大且比较严谨的知识簇，如果文章写的有什么问题，非常欢迎在评论区拷打我。

Read Index

和下一条 Read lease 一样，这是大论文中对 raft 读问题的一个经典优化，不过在讲具体方案前，咱们先看看 raft 在读上面有什么问题，为啥需要这样的优化。

在一个 raft 集群中，一般会有两种读取数据的办法，一种是从任意节点中读取信息，一种是只允许从 Leader 节点中读取。

- 如果以第一种方式读，而又要求读到最新的数据（比如要求[线性一致性](#)），显然是会有问题的。举个简单的例子，假如 Leader 节点提交了一条日志，并且应用到了状态机，而这时某个从节点还不知道这条日志提交了，从这个从节点上的读操作显然是读取不到最新内容的。
- 如果以第二种方式读，也不一保证读取到最新的数据：假如我在当前 Leader 上读时，其它节点已经选出了一个新的 Leader，并且写入了新的内容，而当前旧 Leader 暂时没有感知到这个变化，而是正常执行了读操作，显然读取到的也不是最新的内容。

为了解决这一个问题，原始的 raft 有一个非常简单粗暴的解决方案：**给每一个读操作也生成一条 LogEntry，并且走一趟共识流程**。虽然问题是解决了，但是很显然这个方案在很多应用场景是完全不可接受的，比如【追求性能且只读请求较多】的数据库。

raft 作者当然也发现了这个问题，并且提出了两种比较优雅的方案，其中一种便是 **Read Index**，read index 在处理只读请求时，需要走以下流程（当前假设都是从 Leader 节点读）

1. 如果当前任期内有状态未确认的 LogEntry，先等它提交。假如该 Leader 刚上任且有状态未确认的 LogEntry，则尝试同步一个空 LogEntry 以更新 commit index【no-op】。
2. 经过了第一步，便可以将当前的 **commit index** 暂存起来。
3. 触发一轮新的心跳，如果这轮心跳达成共识，说明：**至少到发送心跳的那个时间点，该节点都是唯一一个被认可的 Leader，所以在那个时间点前暂存的最新 commit index，一定也是整个集群中最新的！**
4. 1-3步确认了一个最新的 commit index，等待这个 index 及其以前的 LogEntry 都应用到状态机。
5. 这样一来，在**发送心跳前**的只读操作，都可以正常执行并且返回结果，它们读到的一定是最新的。（注意这里的“最新的”是从只读请求发起的时间点来定义的，只要完成 1-4 步，即使在读状态机时，有新的 Leader 产生了并且上面有了新的写入操作，都不会影响这些读操作的线性一致性）

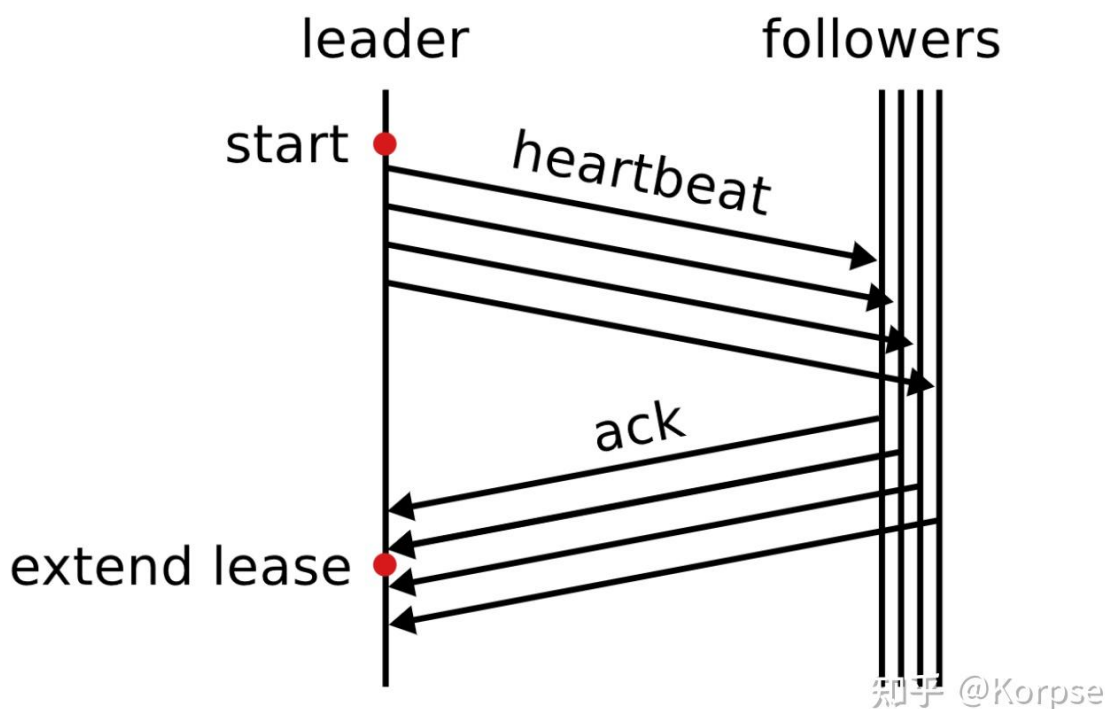
可以看到上面的步骤虽然保证了 Leader 节点上读的线性一致性，但是如果在从节点上进行读取，还是可能会有不一致的问题。但其实解决方案也非常简单：从节点在收到读请求时，向主节点“索要”当前时间的最新 commit index 即可，主节点收到这个“索要”请求后，只需执行 1-3 步便可得到这个 commit index。

参考：

[etcd-raft的线性一致读方法一：ReadIndex - 知乎 \(zhihu.com\)](#)

raft 大论文：6.4 Processing read-only queries more efficiently

Lease Read



Lease 计算方法

Lease Read 同样解决的是 Read Index 解决的问题：如何在线性一致性的要求下提升读请求的性能。

Read Index 虽然和“每个读操作都用 LogEntry 同步”相比，节约了至少一次落盘的时间，但是还是至少要等待一次心跳的共识，也就是至少一个 RTT，这对性能消耗其实还是不可忽略的。

而 Leaser Read 则更机智地消除了这个 RTT 的消耗，它的实现原理甚至比 Read Index 更好理解，可以用一句话来总结：**让主节点算出自己独立掌权的最小时间。**

所以其实现原理就落在了“怎么计算出自己独立掌权的时间？”，raft 把这个独立掌权的时间抽象为了 lease，lease 到期之前，都不会产生新的 Leader。具体的计算方法如下：

1. 主节点正常定期广播心跳
2. 当收到**大部分**心跳回复后，更新 lease。
3. lease 更新为：“发出心跳的时间点”+ election timeout

这个计算方法其实很容易理解：如果大部分节点都收到并且回复了心跳，那么它们至少分别都需要等待一个 election timeout 的时间才会发起选举并且选举出新的 Leader（这样的前提貌似必须是 election timeout 前都不发起选举或者投票？）

但是实际物理机器其实还是有一个问题，对每台机器而言，同样的 election timeout 可能流逝速度有细微的差距，所以实际的 lease 计算方法还要加上一个 clock drift bound：

$$start + \frac{election\ timeout}{clock\ drift\ bound}$$

lease 计算公式

关于 clock drift bound 的定义，可以参照原文

The lease approach assumes a bound on clock drift across servers (over a given time period, no server's clock increases more than this bound times any other)

参考：

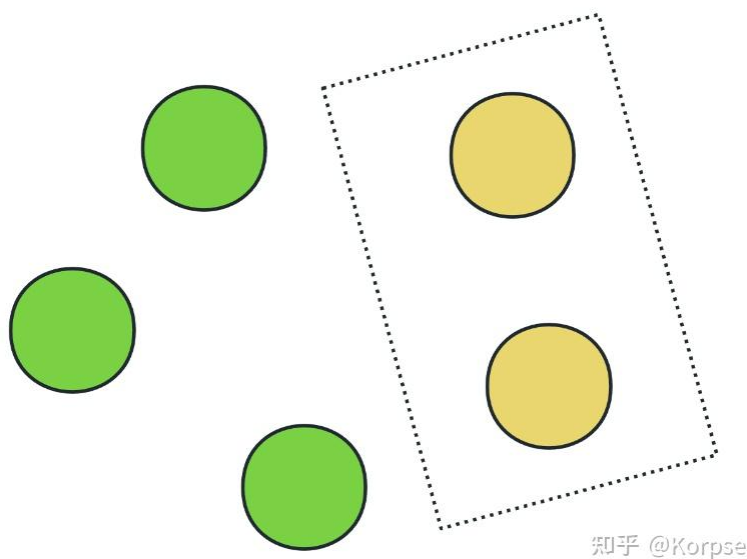
[etcd-raft的线性一致读方法二：LeaseRead - 知乎 \(zhihu.com\)](#)

raft 大论文：6.4.1 Using clocks to reduce messaging for read-only queries

Pre Vote

废话不多说，讲方案前，先来看看它解决了什么问题：

举一个简单的场景：如果集群中少部分节点出现分区（如下图中的黄色节点），由于少了这些节点并不影响 raft 集群的正常允许，所以 Leader 还会照常工作。而分区内的两个节点由于接收不到心跳了，便会一直尝试进行选举，而这些选举由于不可能达成 quorum，并不会真正选出一个 Leader，但是会造成 **term 的不断增加**。



节点分区

那么问题就来了，当它们重新联系上其它节点后，向 Leader 发送的 RequestVote RPC 会造成当前的 Leader 卸任，进入新一轮选举。

然而站在上帝视角的我们非常清楚，Leader 干的好好的，完全没必要卸任并且重新选举嘛。

Pre Vote 解决的就是上述问题，而解决方案也非常直观：我们只需要在正式选举之前尝试进行一轮假的选举，如果这个假的选举能够成功，才说明确实需要进入真正的选举了。

具体流程如下：

1. 某个节点 election timeout 时，不改变 term，而是直接发送 RequestVote 请求，**进行 Pre Vote**
2. 如果：收到大多数节点的选票，则增加 term，进入正式选举，此节点大概率在正式选举时也会当选。

3. 否则：放弃该轮选举，等待下一次 election timeout

虽然解决了 Leader 莫名被卸任的问题，但是 Pre Vote 也有非常明显的缺点：当Leader 节点确实崩溃，需要正式选举时，也要先走一轮 PreVote。加上 raft 选举时可能出现的分区问题，这样的两轮选举让一个新 Leader 产生的时间增加了不少。

参考：[NuRaft/docs/prevote_protocol.md at master · eBay/NuRaft · GitHub](https://github.com/eBay/NuRaft/blob/master/docs/prevote_protocol.md)

Nuraft : Priority election

之前 OB 大佬在我评论区的回复中提到了 raft 的一个致命的缺点：raft 的选举算法的约束非常大（要求成为leader的副本必须具备全部已确认的日志），**它无法实现按照优先级进行选举**。所以该大佬在 OB 的 Leader Election 算法与日志复制完全解耦，可以按照优先级进行选举。

OB 先按下不表，其实 Raft 也是有一些解决这个约束的余地，我在 nuraft 的文档中就发现了这个解决方案：

- 每个节点内置两个参数：
 - `*priority*`：本节点的优先值
 - `*target_priority*`：集群中最大的优先值
- 每次收到心跳时，从节点都确认 target_priority 为最大的优先值
- 但是选举时，只有 priority >= target_priority 的节点才会参选
- **否则**：等待下一轮选举，并将 target_priority * 0.8

可以看出这个方法在一定程度上可以让 raft 按照优先解决进行选举，但是在真正选举时，可能迟迟选不出一个 Leader，这在生产环境中是比较致命的（RTO大大增加）。

参考：[NuRaft/docs/leader_election_priority.md at master · eBay/NuRaft · GitHub](https://github.com/eBay/NuRaft/blob/master/docs/leader_election_priority.md)

Leader change

在实际生产环境下，有一些优化特殊场景下需要 Leader 的主动变更，比如：

1. 主节点要重启，可以主动让出主节点
2. 主节点所在的机器过于繁忙，我们需要迁移到另外一个相对空闲的机器中。
3. 复制组跨IDC部署，希望主节点存在于离Client延时最小的集群中。

Leader change 就是一种主动转让 leader 的实现，除了 raft，[OceanBase](#) 中其实也有对应的类似实现，可以看出这个功能在实际上还是非常必要的。

下面给出的方案是 raft 的一种解决方案：

1. 主停止写入，这时候所有的apply会报错。
2. 继续向所有的follower同步日志，当发现目标节点的日志已经和主一样多之后，向对应节点发起一个TimeoutNow RPC
3. 指定节点收到TimeoutNowRequest之后，直接变为Candidate, 增加term，并开始进入选主
主收到TimeoutNowResponse之后，开始step down.

参考：忘记参考的谁了，俺记得好像是 braft，但是找不到文档了。

Leadership Expiration

同样的，在实际生产环境中，有些问题是需要额外考虑的，一是疑似脑裂问题，Leader以为自己是主，但由于与其他节点断联，其它节点中已经选出了新主，而旧Leader不知情，仍然接受请求但无法同步日志。二也是最关键的问题是失联中非对称网络隔离，例如leader 可以一直发ae 给follower，但是leader 收不到ack，这种问题极其严重，会导致raft 组一直不可用，所以必须自己检测自己。（这段来自评论区大佬指正）

上面两类问题，尤其是第二类，需要一种方法检测出这类故障，并且阻塞新的请求操作。

而几乎所有的系统都必须实现这个功能，比如 mysql 的 MGR，我之前在做测试时，遇到大部分从节点失联的情况，client 的写请求会立即被阻塞。

raft 的解决方案非常简单，不需要引入新的机制，用心跳顺便解决即可：

1. Leader 给每个从节点加上一个超时标签
2. 每次收到从节点对心跳的回复时，更新标签
3. 当一定时间未收到回复，标记其断连
4. 断连从节点达到大多数时，阻塞状态机的输入。

no-op

etcd-raft的线性一致读方法一：ReadIndex

说明

在分布式系统中，存在多种一致性模型，诸如严格一致性、线性一致性、顺序一致性等。不同的一致性模型给应用提供的数据保证也不同，其代价也不一样，一致性越强，代价越高。但是一致性越强，对应用的使用也就越友好。关于一致性模型的更多描述可参考：<https://aphyr.com/posts/313-strong-consistency-models>

问题背景

在分布式系统中，[etcd-raft](#)的实现模式是[Leader + Followers](#)，即存在一个Leader和多个Follower，所有的更新请求都经由Leader处理，Leader再通过日志同步的方式复制到Follower节点。

读请求的处理则没有这种限制：所有的节点（Leader与Followers）都可以处理用户的读请求，但是由于以下几种原因，导致从不同的节点读数据可能会出现不一致：

- Leader和Follower之间存在状态差：这是因为更新总是从Leader复制到Follower，因此，Follower的状态总的落后于Leader，不仅如此，Follower之间的状态也可能存在差异。因此，如果不作特殊处理，从集群不同的节点上读数据，读出的结果可能不相同；
- 假如限制总是从某个特点节点读数据，一般是Leader，但是如果旧的Leader和集群其他节点出现了网络分区，其他节点选出了新的Leader，但是旧Leader并没有感知到新的Leader，于是出现了所谓的[脑裂现象](#)，旧的Leader依然认为自己是主，但是它上面的数据已经是过时的了，如果客户端的请求分别被旧的和新的Leader分别处理，其得到的结果也会不一致。

如果不对读流程作任何的特殊处理，上述限制就会导致一个非一致性的读。而线性一致性的两个要求：**一致性读**和**读最新数据**更是无从谈起。

实现

原理

etcd-raft通过一种称为[ReadIndex](#)的机制来实现线性一致读，其基本原理也很简单：Leader节点在处理读请求时，首先需要与集群多数节点确认自己依然是Leader，然后读取已经被应用到应用状态机的最新数据。

基本原理包含了两方面内容：

- Leader首先通过某种机制确认自己依然是Leader；
- Leader需要给客户端返回最近已应用的数据：即最新被应用到状态机的数据。

数据结构

ReadState

```
type ReadState struct {
    Index      uint64
    RequestCtx []byte
}
```

ReadState负责记录每个客户端的读请求的状态，其中包含：

- ***RequestCtx***：客户端读请求的唯一标识，一般使用request id
- ***Index***：表示读请求产生时当前节点的Commit点

ReadState最终会被返回给应用（通过Ready），由应用负责处理客户端的读请求，而且，应用需要根据该请求发起时的节点Commit信息决定返回何时数据。

readIndexStatus

```
type readIndexStatus struct {
    req  pb.Message
    index uint64
    acks map[uint64]struct{}
}
```

readIndexStatus用来追踪Leader向Followers发送的心跳信息的响应，其中：

- ***req***：表示原始的ReadIndex请求，这是应用在处理客户端读请求时向底层[raft协议](#)核心处理层发起的ReadIndex请求；
- ***index***：表示Leader当前的Commit信息；
- ***acks***：记录Followers的响应，某个Follower如果确认了Leader的心跳消息，acks便会记录一个

readOnly

```

type readOnly struct {
    option          ReadOnlyOption
    pendingReadIndex map[string]*readIndexStatus
    readIndexQueue  []string
}

```

readOnly管理全局的读ReadIndex请求，其中：

- ***option***：暂时不确定含义；
- ***pendingReadIndex***：保存所有的待处理的ReadIndex请求，实现上是一个map，其中key为请求的唯一标识，一般为节点为请求生成的唯一request id；
- ***readIndexQueue***：所有的ReadIndex的数组

关键流程

客户端读请求处理

由于etcd-raft自带的示例应用并没有实现线性一致性读，因此，选择etcd-server作为例子来说明如何通过read index方法来实现线性一致读。

etcd-server在启动时会创建一个后台协程，运行的方法是：*linearizableReadLoop*，如下：

```

func (s *EtcdServer) Start() {
    s.start()
    s.goAttach(func() { s.publish(s.Cfg.ReqTimeout()) })
    s.goAttach(s.purgeFile)
    s.goAttach(func() { monitorFileDescriptor(s.stopping) })
    s.goAttach(s.monitorVersions)
    s.goAttach(s.linearizableReadLoop)
    s.goAttach(s.monitorKVHash)
}

```

这个协程干了什么呢？其实很简单，等着有读请求的信号，并且在有信号来的时候调用底层的raft核心协议处理层来获取信号发生时刻的commit index，如下：

```

func (s *EtcdServer) linearizableReadLoop() {
    var rs raft.ReadState
    for {
        ctx := make([]byte, 8)
        binary.BigEndian.PutUint64(ctx, s.reqIDGen.Next())

        // 主要是等待readwaitc上的信号
        select {
        case <-s.readwaitc:
        case <-s.stopping:
            return
        }

        // 有信号到来
        nextnr := newNotifier()
        s.readMu.Lock()
        nr := s.readNotifier
        s.readNotifier = nextnr
        s.readMu.Unlock()
    }
}

```

```

        // 准备读取当前节点的commit index
        cctx, cancel := context.WithTimeout(context.Background(),
s.Cfg.ReadTimeout())
        if err := s.r.ReadIndex(cctx, ctx); err != nil {
            .....
        }
        cancel()
        // 等待底层返回当次读请求时的commit index
        // 增加了超时控制
        for !timeout && !done {
            .....
        }

        // 判读应用当前已经应用至状态机的index是否小于已commit的index
        // 如果是，等待状态机提交至commit index
        if ai := s.getAppliedIndex(); ai < rs.Index {
            select {
            case <-s.applywait.Wait(rs.Index):
            case <-s.stopping:
                return
            }
        }
        // 至此,commit index已经被应用至状态机
        // 可以通知读请求从状态机中返回数据了
        nr.notify(nil)
    }
}

```

这里的linearizableReadLoop相当于是把一把锁，控制读请求何时可以从状态机中读取最新数据。

我们上面说到linearizableReadLoop是在收到信号的时候进行read index控制，那该信号又是由谁触发呢？

信号的直接来源是linearizableReadNotify:

```

func (s *EtcdServer) linearizableReadNotify(ctx context.Context) error {
    s.readMu.RLock()
    nc := s.readNotifier
    s.readMu.RUnlock()

    // 这里发送信号
    select {
    case s.readwaitc <- struct{}{}:
    default:
    }

    // 等待read state结束
    select {
    case <-nc.c:
        return nc.err
    case <-ctx.Done():
        return ctx.Err()
    case <-s.done:
        return ErrStopped
    }
}

```

```
}
```

linearizableReadNotify的实现非常简单，这里就不再赘述，linearizableReadNotify会在读请求中被调用，如下：

```
func (s *EtcdServer) Range(ctx context.Context, r *pb.RangeRequest)
(*pb.RangeResponse, error) {
    if !r.Serializable {
        err := s.linearizableReadNotify(ctx)
        if err != nil {
            return nil, err
        }
    }
    .....
}
```

上面我们就以etcd-server为例分析了客户端的读请求是如何在服务端被处理。由于增加了read index过程而导致了比正常的请求处理增加了复杂性。接下来，我们要看看raft的核心协议处理层如何处理这种ReadIndex请求。

核心协议层处理ReadIndex请求

也即：客户端的读请求也需要被参与到协议的核心处理层。为此，核心处理层提供了一个接口专门处理读，ReadIndex：

```
func (n *node) ReadIndex(ctx context.Context, rctx []byte) error {
    return n.step(ctx, pb.Message{Type: pb.MsgReadIndex, Entries:
[]pb.Entry{{Data: rctx}}})
}

func (n *node) step(ctx context.Context, m pb.Message) error {
    ch := n.revcv
    if m.Type == pb.MsgProp {
        ch = n.propc
    }
    select {
    case ch <- m:
        return nil
    case <-ctx.Done():
        return ctx.Err()
    case <-n.done:
        return ErrStopped
    }
}
```

ReadIndex的消息最终被发往了recvc这个channel，后台协程会接管这个消息并调用核心的协议处理函数：

```

func (n *node) run(r *raft) {
    for {
        select {
        case m := <-n.recv:
            _, ok := r.prs[m.From]
            if ok || !IsResponseMsg(m.Type) {
                r.Step(m)
            }
        case ...:
        }
    }
}

```

如果节点是Leader，那么该消息最终的处理函数是stepLeader:

```

func stepLeader(r *raft, m pb.Message) {
    switch m.Type {
    case pb.MsgReadIndex:
        if r.quorum() > 1 {
            if
r.raftLog.zeroTermOnErrCompacted(r.raftLog.term(r.raftLog.committed)) != r.Term {
                return
            }
            switch r.readOnly.option {
            // 关注的是该分支
            case ReadOnlySafe:
                r.readOnly.addRequest(r.raftLog.committed, m)
                r.bcastHeartbeatWithCtx(m.Entries[0].Data)
            case ReadOnlyLeaseBased:
                .....
            }
        }
    }
}

```

本文讨论的ReadIndex方案对应的是ReadOnlySafe这个分支，其中addRequest()会把这个读请求到达时的leader的commit index保存起来，并且维护一些状态信息，而bcastHeartbeatWithCtx()则向其他Followers节点发送心跳消息MsgHeartbeat，而当leader收到心跳响应消息MsgHeartbeatResp时处理为:

```

func stepLeader(r *raft, m pb.Message) {
    .....
    case pb.MsgHeartbeatResp:
        ackCount := r.readOnly.recvAck(m)
        if ackCount < r.quorum() {
            return
        }
        rss := r.readOnly.advance(m)
        for _, rs := range rss {
            req := rs.req
            if req.From == None || req.From == r.id {
                // from local member
                r.readStates = append(r.readStates, ReadState{Index: rs.index,
RequestCtx: req.Entries[0].Data})
            } else {

```

```

        // 如果是follower节点发来的ReadIndex请求
        // 给它返回响应
        r.send(pb.Message{To: req.From, Type: pb.MsgReadIndexResp, Index:
rs.index, Entries: req.Entries})
    }
}
}
}

```

如果接收到了多数派的心跳响应，则会从刚才保存的信息中将对应读请求当时的commit index和请求id拿出来，填充到ReadState中，该结构最终会被返回给调用ReadIndex的应用。

需要说明的是：r.readOnly.advance(m)会返回m以前的所有的request id（m其实也就是应用调用ReadIndex时的请求id，而且请求id是单调递增的，因此当在某个时刻确认了请求id为m的主信息，那么m之前的主信息也认为是此），因此，上面代码的rss其实是一个slice。

以上我们其实是假设该读请求是发生在leader上面的，leader通过心跳确认自己是主，然后再保证读时刻的commit index被应用至状态机后返回读结果给客户端，这就保证了客户端得到的结果一定是最新的。

那假如读请求是发生在follower上呢？对于etcd-server应用来说，它照样是遵循上面的请求处理流程，应用层会调用协议的核心处理层的ReadIndex方法，但是核心协议层识别当前节点是follower，那请求处理流程是：

```

func stepFollower(r *raft, m pb.Message) {
    switch m.Type {
        .....
    case pb.MsgReadIndex:
        if r.lead == None {
            return
        }
        m.To = r.lead
        r.send(m)
    case pb.MsgReadIndexResp:
        r.readStates = append(r.readStates, ReadState{Index: m.Index, RequestCtx:
m.Entries[0].Data})
        .....
    }
}

```

可以发现，如果是在Follower节点上执行ReadIndex，那么它必须先要向Leader去查询commit index，然后收到响应后在创建ReadState记录commit信息，后续的处理和Leader别无二致。

总结

经过上面一通复杂处理，就达到了效果：无论从主还是从上去读，保证读到的数据都一致（都是在主上被commit后的状态）

参考

- <https://zhuanlan.zhihu.com/p/27>

etcd-raft的线性一致读方法二：LeaseRead

说明

我们在前面的ReadIndex中说明了[etcd-raft](#)如何通过ReadIndex的思路来实现线性一致性读。虽然在读的时候只会交互Heartbeat信息，这毕竟还是有代价，所以我们可以考虑做进一步的优化。

在 Raft 论文里面，提到了一种通过 clock + heartbeat 的 lease read [优化方法](#)。leader在发送 heartbeat 的时候，会首先记录一个时间点 start，当系统大部分节点都回复了 heartbeat response，那么我们就可以认为 leader 的 lease 有效期可以到 start + election timeout / clock drift bound 这个时间点。

为什么能够这么认为呢？主要是在于 Raft 的选举机制，因为 follower 会在至少 election timeout 的时间之后，才会重新发生选举，所以下一个 leader 选出来的时间一定可以保证大于 start + election timeout / clock drift bound。

虽然采用 lease 的做法很高效，但仍然会面临风险问题，也就是我们有了一个预设的前提，各个服务器的 CPU clock 的时间是准的，即使有误差，也会在一个非常小的 bound 范围里面，如果各个服务器之间 clock 走的频率不一样，有些太快，有些太慢，这套 lease 机制就可能出问题。

实现

etcd-server对客户的读请求处理与前面介绍的ReadIndex方法别无二致，区别在于对ReadIndex请求的处理，两种不同的方式走了不同的路径：

```
func stepLeader(r *raft, m pb.Message) {
    case pb.MsgReadIndex:
        if r.quorum() > 1 {
            switch r.readonly.option {
                case ReadOnlySafe:
                    .....
                // read lease机制
                case ReadOnlyLeaseBased:
                    var ri uint64
                    if r.checkQuorum {
                        ri = r.raftLog.committed
                    }
                    if m.From == None || m.From == r.id {
                        r.readStates = append(r.readStates,
                            ReadState{
                                Index: r.raftLog.committed,
                                RequestCtx: m.Entries[0].Data})
                    } else {
                        r.send(pb.Message{To: m.From,
                            Type: pb.MsgReadIndexResp,
                            Index: ri,
                            Entries: m.Entries})
                    }
                }
            }
            ...
        }
    }
}
```

如果是使用ReadOnlyLeaseBased选项（即read lease机制），如果ReadIndex请求是来自当前节点的应用，主节点的处理函数就直接将自己的当前commit返回给应用即可，而如果ReadIndex请求是来自集群其他Follower节点，给Follower节点返回MsgReadIndexResp，其中包含Leader的[commit index](#)。

对于Follower节点，如果收到应用的ReadIndex请求，无论是ReadIndex还是LeaseRead做法都一样，直接发送ReadIndex请求给Leader即可。

在LeaseRead方案中，主节点之所以可以直接将自身的commit index信息返回是因为其认为自身当前依然持有作为Leader的尚方宝剑（lease）。为此，Leader节点需要定期检查自身是否依然是主。

```
func (r *raft) tickHeartbeat() {
    .....
    if r.electionElapsed >= r.electionTimeout {
        r.electionElapsed = 0
        if r.checkQuorum {
            r.Step(pb.Message{From: r.id,
                Type: pb.MsgCheckQuorum})
        }
        if r.state == StateLeader && r.leadTransferee != None {
            r.abortLeaderTransfer()
        }
    }
    .....
}
```

Leader在发送心跳的时候会进行Quorum检查，但不是每次心跳都会检查，只要当前距离上一次选举的时间间隔不超过系统的electionTimeout时间，我们就认为在该安全周期内不会发生新的选举（在前面已经说明）。

检查的原理是发送消息MsgCheckQuorum给自己（注意：该消息是由Leader发送给自己，提醒自己进行Leader身份检查），该消息处理是：

```
func stepLeader(r *raft, m pb.Message) {
    switch m.Type {
    case pb.MsgBeat:
        ...
    case pb.MsgCheckQuorum:
        if !r.checkQuorumActive() {
            r.becomeFollower(r.Term, None)
        }
        return
    case ...:
    }
}

// 检查多数节点是否存活
func (r *raft) checkQuorumActive() bool {
    var act int
    for id := range r.prs {
        if id == r.id { // self is always active
            act++
            continue
        }

        if r.prs[id].RecentActive {
```

```

        act++
    }
    r.prs[id].RecentActive = false
}
return act >= r.quorum()
}

```

主要检查在于函数`checkQuorumActive`，该函数会检查多数从节点是否活跃，根据`RecentActive`标识位来判断，同时在检查后将该标志位设置为`false`，避免出现每次检查都是存活，出现状态错误。

而`RecentActive`标志位是Leader节点在收到Follower的消息响应后被设置为`true`的，这些响应包括心跳响应（`MsgHeartbeatResp`）与正常的数据同步响应（`MsgAppResp`）。

```

func stepLeader(r *raft, m pb.Message) {
    ...
    switch m.Type {
    case pb.MsgAppResp:
        pr.RecentActive = true
        .....
    case pb.MsgHeartbeatResp:
        pr.RecentActive = true
    }
    ...
}

```

这些响应保证了Follower任然认可其Leader的身份，而在`checkQuorumActive`检查是否多数Follower依然认可Leader的领导者身份，从而保证了Leader当前对lease的持有，并且可以不经任何确认就可以直接返回自身的commit作为ReadIndex的响应。

参考

- <https://zhuanlan.zhihu.com/p/25>